Smart Glasses Agent PoC Final Architecture Document

1. Document Purpose

This document consolidates the previously prepared architecture notes for smart-glasses-agent-poc into a single final reference. It is intended to serve as the primary architecture document for:

- system overview
- apps/api code walkthrough
- service responsibilities
- user request data flows
- user scenarios
- Mermaid sequence diagrams
- functional block diagrams
- baseline versus optimized comparison

The focus is the actual implemented PoC, not a hypothetical target architecture.

---

2. System Summary

smart-glasses-agent-poc is a smart-glasses-style Physical AI Agent PoC. It accepts multimodal context from:

- user request text
- image or video
- GPS context
- nearby device metadata

The backend routes the request to an appropriate service, optionally performs device action execution, reuses prior context with a lightweight GraphRAG layer, and records evaluation metrics for baseline versus optimized comparison.

Main services

1. Scene Assistant
2. Navigation
3. Device Control
4. Safety Alert
5. Context Memory
6. Label Reader

Main optimization ideas

- reduced video processing through frame sampling
- query-aware keyframe selection

- semantic compression before cloud inference
- rule-based router with VLM fallback
- graph and vector context retrieval
- per-request evaluation logging

---

3. Repository Focus

This document focuses mainly on apps/api, with only the minimum frontend references required to explain end-to-end flow.

Core backend areas

- apps/api/app/main.py
- apps/api/app/config.py
- apps/api/app/groq_client.py
- apps/api/app/api/*
- apps/api/app/agent/*
- apps/api/app/perception/*
- apps/api/app/services/*
- apps/api/app/actions/*
- apps/api/app/memory/*
- apps/api/app/evaluation/*
- apps/api/app/schemas/*
- apps/api/app/demo/seed.py

Supporting frontend areas

- apps/web/src/pages/PocDashboard.tsx
- apps/web/src/components/*
- apps/web/src/api/agentApi.ts

---

4. High-Level Architecture

Logical layers

1. Input layer
2. API layer
3. Pipeline orchestration layer
4. Perception layer
5. Retrieval layer
6. Routing layer
7. Service layer

8.Action and policy layer

9.Memory layer

10.Evaluation layer

11.Frontend visualization layer

Functional role of each layer

- Input layer collects multimodal request data.
- API layer parses multipart request and delegates to the planner.
- Planner coordinates the entire run.
- Perception converts media into keyframes, base64 payloads, and semantic prompts.
- Retrieval reuses prior graph/vector memory.
- Router chooses a service using rules first, then VLM fallback if needed.
- Service layer produces the actual user-facing response.
- Action layer enforces device capability and safety policy.
- Memory layer stores and retrieves scene summaries.
- Evaluation layer records cost and latency evidence.
- Frontend surfaces the agent output, graph state, and performance metrics.

---

5. API Entry and Request Contract

FastAPI entrypoint

apps/api/app/main.py

The application registers:

- /api/agent
- /api/context
- /api/graph
- /api/logs
- /health
- /api/demo/reset — ■■ ■■ ■■ (■■■ ■■■ + ■■ ■■ + ■■ ■■■)

On startup, lifespan calls seed_demo_memory() to pre-populate demo context via apps/api/app/demo/seed.py.

Main execution endpoint

apps/api/app/api/agent.py

The endpoint:

- receives context_json through multipart form data
- optionally receives image
- optionally receives video
- parses context_json into ContextRequest

- reads uploaded bytes
- calls run_pipeline(image_bytes, video_bytes, ctx)

Request model

apps/api/app/schemas/context.py

Key request fields:

- user_request
- gps
- nearby_devices
- mode

Response model

apps/api/app/schemas/agent.py

Key response fields:

- request_id
- selected_service
- router_confidence
- vlm_used
- response_text
- action_result
- original_frame_count
- selected_keyframe_count
- retrieved_graph_nodes
- latency_ms
- mode

Evaluation log model

apps/api/app/schemas/log.py

Additional persisted metrics:

- timestamp (auto-filled datetime.utcnow)
- user_request
- token_count
- image_payload_bytes — 3-way logic: pure semantic bytes / semantic+vision fallback / raw image bytes
- cloud_called (alias: vlm_used)
- fallback_reason
- failure_type — populated by classify_failure() in planner

latency_ms is a LatencyBreakdown struct (not a flat int), shared between AgentResponse and EvaluationLog.

---

6. Core Pipeline Walkthrough

Orchestrator

apps/api/app/agent/planner.py

run_pipeline() is the central coordinator.

Execution stages

1.create request_id

2.run perception

3.(optimized only) start GraphRAG retrieval and routing concurrently via asyncio.create_task — retrieval wall-clock cost is hidden behind routing

4.(baseline) run routing sequentially

5.inject graph_context into semantic_prompt after retrieval completes

6.execute the selected service

7.persist memory in optimized mode (context_memory responses are excluded to prevent recursive contamination)

8.write evaluation log

9.return AgentResponse

Internal control points

•perception happens before retrieval and routing

•retrieval result is injected into semantic prompt after concurrent routing+retrieval both complete

•router result and service result both contribute to usage metrics

•graph/vector memory writes happen only in optimized mode

•context_memory service response is NOT stored in graph/vector store (prevents recursive contamination)

•evaluation metrics are collected even for baseline mode

---

7. Perception Architecture

Main files

•app/agent/pipeline_support.py

•app/perception/frame_sampler.py

•app/perception/keyframe_selector.py

•app/perception/image_preprocessor.py

•app/perception/semantic_extractor.py

Image path

For image input:

•decode image with OpenCV

•preprocess to resized JPEG base64

- if optimized, extract semantic features
- build semantic prompt

Video path

For video input:

- write video bytes to a temporary file
- sample frames
- if optimized, reduce fps and run query-aware keyframe selection
- encode selected frames to base64
- if optimized, extract semantic payloads from keyframes

Semantic extraction outputs

- scene brightness
- motion level
- dominant colors
- OCR text if available

Why the perception layer matters

This is where optimized mode diverges most strongly from baseline. The system attempts to reduce cloud payload size and preserve only task-relevant visual information before service execution.

---

8. Retrieval and GraphRAG Architecture

Main files

- app/memory/graph_store.py
- app/memory/vector_store.py
- app/memory/retrieval.py

Memory design

The PoC uses a hybrid memory model:

- graph memory for structured scene context
- vector memory for coarse semantic retrieval

Graph node types

- Scene
- Object
- Location
- Device
- UserIntent
- Action

- Risk
- Time

Graph retrieval behavior

- score scene nodes by query overlap
- expand neighboring nodes using BFS
- collect objects, actions, risks, and location
- return context strings

Vector retrieval behavior

- uses Qdrant if reachable
- otherwise uses in-memory fallback
- demo embedding is a deterministic pseudo-vector based on character frequency

Hybrid retrieval output

retrieve_context() merges:

- vector hits
- graph hits

The planner converts the merged result to graph_context for downstream service prompts.

---

9. Routing Architecture

Main files

- app/agent/router.py
- app/agent/pipeline_support.py

Rule-based routing

The router uses keyword categories to classify:

- safety
- device control
- navigation
- context memory
- scene understanding

Routing signals

- user request keywords
- gps.location_type
- nearby device presence

Fallback behavior

If confidence is below threshold:

- a text-only VLM classification prompt is sent
- the returned string is parsed into a known service name
- fallback reason is recorded

Design intent

The default path is cheap and deterministic. Cloud classification is only used when local routing is uncertain.

---

10. Service Layer Architecture

Service registry

app/agent/service_registry.py

Maps (6 services):

- scene_assistant
- navigation
- device_control
- safety_alert
- context_memory
- label_reader

Unknown service names fall back to scene_assistant.run via get_service_runner().

Common service dispatch logic

app/services/common.py

Shared capabilities:

- raw VLM execution
- semantic-first VLM execution
- semantic fallback to image vision call
- usage merging

Service responsibilities

Scene Assistant

- describes what is visible
- handles general scene questions
- uses image or semantic prompt

Navigation

- uses GPS context
- may reuse prior graph context
- returns short practical guidance

Device Control

- selects a target device from metadata
- infers action from request
- runs mock execution with policy enforcement
- typically does not use VLM

Safety Alert

- analyzes hazards and caution points
- sanitizes unsafe certainty language
- stores risk-related memory hints

Context Memory

- reuses planner retrieval if available
- otherwise triggers retrieval itself
- asks a text model to answer using stored past context

Label Reader

- reads medicine labels, product packaging, or signage
- OCR-present path: text-only VLM from semantic prompt
- OCR-empty path: direct vision call (bypasses text-only path)
- clearest PoC example of semantic compression value

---

11. Action and Policy Architecture

Main files

- app/actions/device_registry.py
- app/actions/executor.py
- app/agent/policy.py

Device capability registry

Supported types include:

- smart light
- speaker
- TV
- air conditioner
- door lock

Each capability includes:

- supported actions
- risk level
- whether confirmation is required

Action execution flow

1.look up capability

2.infer action from user request

3.run policy check

4.return mock result

Safety guardrails

Policy blocks:

•dangerous or unsupported actions

•high-risk operations requiring user confirmation

•overconfident safety wording in textual responses

---

12. Evaluation Architecture

Main files

•app/evaluation/logger.py

•app/evaluation/metrics.py

Logged metrics

•frame sampling latency

•keyframe selection latency

•retrieval latency

•routing latency

•VLM latency

•total latency

•selected service

•VLM call count

•graph retrieval count

•token count

•image payload bytes

•fallback reason

•failure type

Why this layer exists

The PoC is intended not only to answer requests but also to demonstrate the effect of optimization strategies compared with baseline mode.

---

13. Frontend Integration

Main frontend roles

- submit request payload
- display selected service and confidence
- display textual response and action result
- display performance metrics
- display graph memory state

Frontend positioning after UI redesign

The frontend should now be understood as a presentation-oriented operator console, not as the final in-glasses end-user interface.

It is optimized for:

- live demo control
- optimization evidence presentation
- pipeline explainability
- side-by-side baseline versus optimized storytelling

It is not optimized for:

- low-vision or blind user direct operation
- wearable-sized interaction surfaces
- voice-first interaction
- production accessibility polish

Backend APIs used by frontend

- POST /api/agent/run
- GET /api/logs/metrics
- GET /api/graph/nodes
- GET /api/graph/query
- GET /api/graph/size
- POST /api/demo/reset (demo presenter control)

Frontend hooks

- src/hooks/useTTS.ts — Web Speech API TTS hook, used to read out response_text aloud

Dashboard result surfaces

- AgentDecisionPanel
- OutputPanel
- PerformancePanel
- GraphContextPanel
- hero summary metrics

Updated frontend panel intent

PocDashboard

- acts as the main operator stage
- frames the demo around optimization evidence rather than generic dashboard output
- separates mission setup, decision trace, output, proof board, and memory layer

InputPanel

- acts as a scenario director
- prioritizes quick scenario loading, execution mode choice, and payload staging
- is intended to help the presenter drive repeatable demo flows

AgentDecisionPanel

- acts as the pipeline trace panel
- explains selected service, confidence, frame reduction, and latency composition
- is meant to make the orchestration path legible to reviewers

OutputPanel

- acts as the audience-visible result panel
- shows what the user would hear or what device action would occur
- uses a structured rendering path for label_reader

PerformancePanel

- acts as the optimization evidence board
- emphasizes proof of improvement instead of generic telemetry
- should be interpreted as the main panel for payload and latency comparison

GraphContextPanel

- acts as the GraphRAG memory surface
- remains list-oriented, but now also functions as a memory snapshot panel for demos
- is intended to expose accumulated memory state across runs

Frontend design direction

The current visual system uses a telemetry-console aesthetic:

- dark atmospheric background
- glass-like layered cards
- compact uppercase metadata labels
- emphasized hero metrics
- section framing for presenter storytelling

This direction is appropriate for technical presentations and PoC evaluation, because it highlights:

- system stages
- optimization claims
- evidence panels
- backend observability

However, it should not be mistaken for a production companion interface for visually impaired end users.

---

14. Service-by-Service User Scenarios

Scene Assistant

Typical use cases:

- describe a new indoor environment
- summarize a short walking video

Expected outcome:

- concise, direct scene summary
- object-level awareness without heavy over-claiming

Navigation

Typical use cases:

- ask for current route guidance
- ask how to return to a place seen earlier

Expected outcome:

- short directions grounded in GPS and optional prior context

Device Control

Typical use cases:

- turn off a smart light
- raise speaker volume
- attempt a blocked high-risk action like unlocking a door

Expected outcome:

- deterministic local action handling
- clear denial when risk policy blocks execution

Safety Alert

Typical use cases:

- ask whether a crosswalk is safe
- ask whether a dim street contains visible hazards

Expected outcome:

- caution-oriented response
- no strong guarantee of safety

Context Memory

Typical use cases:

- ask which cafe was seen earlier

- ask what past location was noticed
- ask for prior context when no matching memory exists

Expected outcome:

- retrieval-backed answer when memory exists
- explicit memory miss response when it does not

---

15. Common Data Flow

Top-level request flow

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request]
    FE[Frontend Form Submission]
    API[POST /api/agent/run]
    CTX[ContextRequest Parsing]
    PL[planner.run_pipeline]
    PER[Perception]
    RET[Graph Retrieval optimized only]
    ROU[Service Routing]
    SVC[Selected Service Execution]
    ACT[Optional Action Execution]
    MEM[Optional Memory Persistence optimized only]
    LOG[Evaluation Log Write]
    RES[AgentResponse]
    UI[Frontend Panels Update]

    U --> FE --> API --> CTX --> PL --> PER
    PER --> RET
    PER --> ROU
    RET --> SVC
    ROU --> SVC
    SVC --> ACT --> MEM --> LOG --> RES --> UI

    %% Note: in optimized mode, RET and ROU run concurrently (asyncio.create_task)
```

Common data elements

Input data:

- user_request
- gps
- nearby_devices
- mode
- image_bytes or video_bytes

Intermediate data:

- image_b64_list
- semantic_prompt
- graph_context
- routing_result
- service_vlm_usage

Output data:

- selected_service

- response_text
- action_result
- latency_ms

---

16. Service-Specific Data Flows

Scene Assistant

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request What do you see here]
    M[image or video]
    API[/api/agent/run]
    P[run_perception]
    R[route_service]
    S[scene_assistant.run]
    D[dispatch]
    T[run_semantic_service optimized]
    V[run_vlm_service baseline]
    X[response_text]
    G[graph_store.add_scene]
    L[logger.write_log]
    O[AgentResponse]

    U --> API
    M --> API
    API --> P --> R --> S --> D
    D --> T --> X
    D --> V --> X
    X --> G --> L --> O
```

Navigation

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request navigation]
    G0[GpsContext]
    API[/api/agent/run]
    P[run_perception]
    RET[retrieve_context]
    R[route_service]
    N[navigation.run]
    GPS[gps_info]
    PR[prompt build]
    D[dispatch]
    RESP[response_text]
    MEM[memory write]
    LOG[evaluation log]
    OUT[AgentResponse]

    U --> API
    G0 --> API
    API --> P --> RET --> R --> N
    G0 --> GPS --> PR
    RET --> PR
    N --> PR --> D --> RESP --> MEM --> LOG --> OUT
```

Device Control

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request device control]
    DEV[nearby_devices]
    API[/api/agent/run]
    P[run_perception]
    R[route_service]
    S[device_control.run]
```

```
    T[Select target device]
    C[get_capability]
    A[infer_action]
    POL[check_action_allowed]
    EX[mock execute_device_action]
    RES[action_result plus response_text]
    LOG[evaluation log]
    OUT[AgentResponse]

    U --> API
    DEV --> API
    API --> P --> R --> S --> T --> C --> A --> POL
    POL -->|allowed| EX --> RES
    POL -->|blocked| RES
    RES --> LOG --> OUT
```

Safety Alert

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request safety]
    GPS[GpsContext]
    M[image or video]
    API[/api/agent/run]
    P[run_perception]
    K[keyframe selection]
    S0[semantic extraction]
    RET[retrieve_context]
    R[route_service]
    S[safety_alert.run]
    D[dispatch]
    VLM[VLM response]
    SAN[sanitize_safety_response]
    MEM[memory write]
    LOG[evaluation log]
    OUT[AgentResponse]

    U --> API
    GPS --> API
    M --> API
    API --> P --> K --> S0 --> RET --> R --> S --> D --> VLM --> SAN --> MEM --> LOG --> OUT
```

Context Memory

[Diagram — Mermaid source]

```
flowchart TD
    U[User Request memory]
    API[/api/agent/run]
    P[run_perception optional]
    RET[retrieve_context]
    VS[vector_store.search]
    GS[graph_store.find_subgraph_by_query]
    COMB[combined retrieval]
    R[route_service]
    CM[context_memory.run]
    PROMPT[memory prompt]
    TXT[text-only VLM]
    RESP[response_text]
    LOG[evaluation log]
    OUT[AgentResponse]

    U --> API --> P --> RET
    RET --> VS --> COMB
    RET --> GS --> COMB
    COMB --> R --> CM --> PROMPT --> TXT --> RESP --> LOG --> OUT
```

---

17. Sequence Diagrams

Common top-level sequence

[Diagram — Mermaid source]

```
sequenceDiagram
    participant U as User
    participant FE as Frontend
    participant API as /api/agent/run
    participant PL as planner.run_pipeline
    participant PER as Perception
    participant RET as Retrieval
    participant ROU as Router
    participant SVC as Service
    participant MEM as Memory
    participant LOG as Evaluation Logger

    U->>FE: request + image/video + gps + devices
    FE->>API: multipart form submission
    API->>PL: ContextRequest + media bytes
    PL->>PER: run_perception()
    PER-->>PL: image_b64_list, semantic_prompt, metrics
    PL->>RET: retrieve_context() optimized only
    RET-->>PL: graph_context
    PL->>ROU: route_service()
    ROU-->>PL: selected service
    PL->>SVC: run(...)
    SVC-->>PL: response_text, action_result, usage
    PL->>MEM: memory write optimized only
    PL->>LOG: write_log()
    PL-->>API: AgentResponse
    API-->>FE: JSON response
```

Device control policy sequence

[Diagram — Mermaid source]

```
sequenceDiagram
    participant DEV as device_control
    participant ACT as Action Executor
    participant REG as Device Registry
    participant POL as Policy

    DEV->>ACT: execute_device_action(target, user_request)
    ACT->>REG: get_capability(device.type)
    REG-->>ACT: capability
    ACT->>REG: infer_action(user_request, supported_actions)
    REG-->>ACT: action
    ACT->>POL: check_action_allowed(action, risk, confirmation)
    POL-->>ACT: allowed or blocked
    ACT-->>DEV: ActionResult
```

Semantic fallback sequence

[Diagram — Mermaid source]

```
sequenceDiagram
    participant SVC as Service Common
    participant VLM as Groq Client

    SVC->>VLM: text-only semantic prompt
    VLM-->>SVC: semantic response
    alt response too short
        SVC->>VLM: image + fallback vision prompt
        VLM-->>SVC: fallback response
    end
```

---

18. Functional Block Diagrams

Whole system

[Diagram — Mermaid source]

```
flowchart LR
    U[User]
    FE[Frontend Console]
```

```
    API[FastAPI API Layer]
    ORCH[Pipeline Orchestrator]
    PER[Perception Layer]
    RET[Context Retrieval Layer]
    ROU[Routing Layer]
    SVC[Service Layer]
    ACT[Action Layer]
    MEM[Memory Layer]
    VLM[Groq Model Access]
    EVAL[Evaluation Layer]

    U --> FE --> API --> ORCH
    ORCH --> PER
    ORCH --> RET
    ORCH --> ROU
    ORCH --> SVC
    SVC --> ACT
    RET --> MEM
    ORCH --> MEM
    SVC --> VLM
    ROU --> VLM
    ORCH --> EVAL
```

Request handling blocks

[Diagram — Mermaid source]

```
flowchart LR
    A[User Input]
    B[Request Parsing]
    C[Perception Processing]
    D[Context Retrieval]
    E[Service Routing]
    F[Selected Service Execution]
    G[Action Handling optional]
    H[Memory Persistence optimized only]
    I[Evaluation Logging]
    J[Agent Response]

    A --> B --> C --> D --> E --> F --> G --> H --> I --> J
```

Memory architecture blocks

[Diagram — Mermaid source]

```
flowchart LR
    Q[Current User Request]
    RET[Retrieval Controller]
    VQ[Vector Search]
    GQ[Graph Subgraph Query]
    COMB[Combined Context]
    OUT[graph_context]

    Q --> RET
    RET --> VQ
    RET --> GQ
    VQ --> COMB
    GQ --> COMB
    COMB --> OUT
```

---

19. Baseline Versus Optimized Comparison

Baseline path

- no semantic compression
- no graph retrieval
- no memory persistence
- usually raw image or raw text VLM path

- evaluation still recorded

Optimized path

- reduced fps for video
- query-aware keyframe selection
- semantic prompt generation
- graph and vector retrieval
- semantic-first VLM path
- optional vision fallback
- graph and vector memory persistence
- evaluation recorded with richer evidence

Comparison diagram

[Diagram — Mermaid source]

```
flowchart LR
    REQ[Same Request]

    subgraph B[Baseline Path]
        B1[Perception]
        B2[No semantic compression]
        B3[No retrieval]
        B4[Rule routing]
        B5[Raw VLM path]
        B6[Evaluation only]
    end

    subgraph O[Optimized Path]
        O1[Perception]
        O2[Reduced fps]
        O3[Keyframe selection]
        O4[Semantic prompt]
        O5[Graph plus vector retrieval]
        O6[Rule routing]
        O7[Semantic-first VLM path]
        O8[Memory persistence]
        O9[Evaluation]
    end

    REQ --> B1
    REQ --> O1
```

---

20. Testing Model

Main test file

apps/api/tests/test_agent.py

Shared test fixtures

apps/api/tests/conftest.py

Test design highlights

- resets graph and vector state before each test
- redirects log output to a temporary directory
- stubs VLM calls for deterministic integration behavior

- validates endpoint-level behavior rather than low-level algorithm details

Important verified paths

- text-only request handling
- valid service selection
- device control action result
- baseline mode handling
- image upload handling

---

21. Strengths and Current Limits

Strengths

- clear orchestration through planner.py
- visible separation between baseline and optimized behavior
- local fallback paths avoid hard failure during development
- services share a common dispatch pattern
- memory, routing, and evaluation are observable and testable

Limits

- query-aware relevance scoring is heuristic
- pseudo-vector embedding is not production-grade
- device action inference is keyword based
- graph memory is summary-oriented rather than full perception graph
- Groq calls are wrapped in an async function but underlying client behavior is effectively synchronous

---

22. Recommended Reading Order for Engineers

1. apps/api/app/main.py
2. apps/api/app/config.py
3. apps/api/app/groq_client.py
4. apps/api/app/api/agent.py
5. apps/api/app/agent/planner.py
6. apps/api/app/agent/pipeline_support.py
7. apps/api/app/agent/router.py
8. apps/api/app/services/common.py
9. apps/api/app/services/safety_alert.py
10. apps/api/app/services/device_control.py
11. apps/api/app/memory/graph_store.py

12.apps/api/app/memory/retrieval.py

13.apps/api/app/evaluation/logger.py

14.apps/api/tests/conftest.py

---

23. Source Notes Consolidated Into This Document

This final document consolidates the content previously captured in:

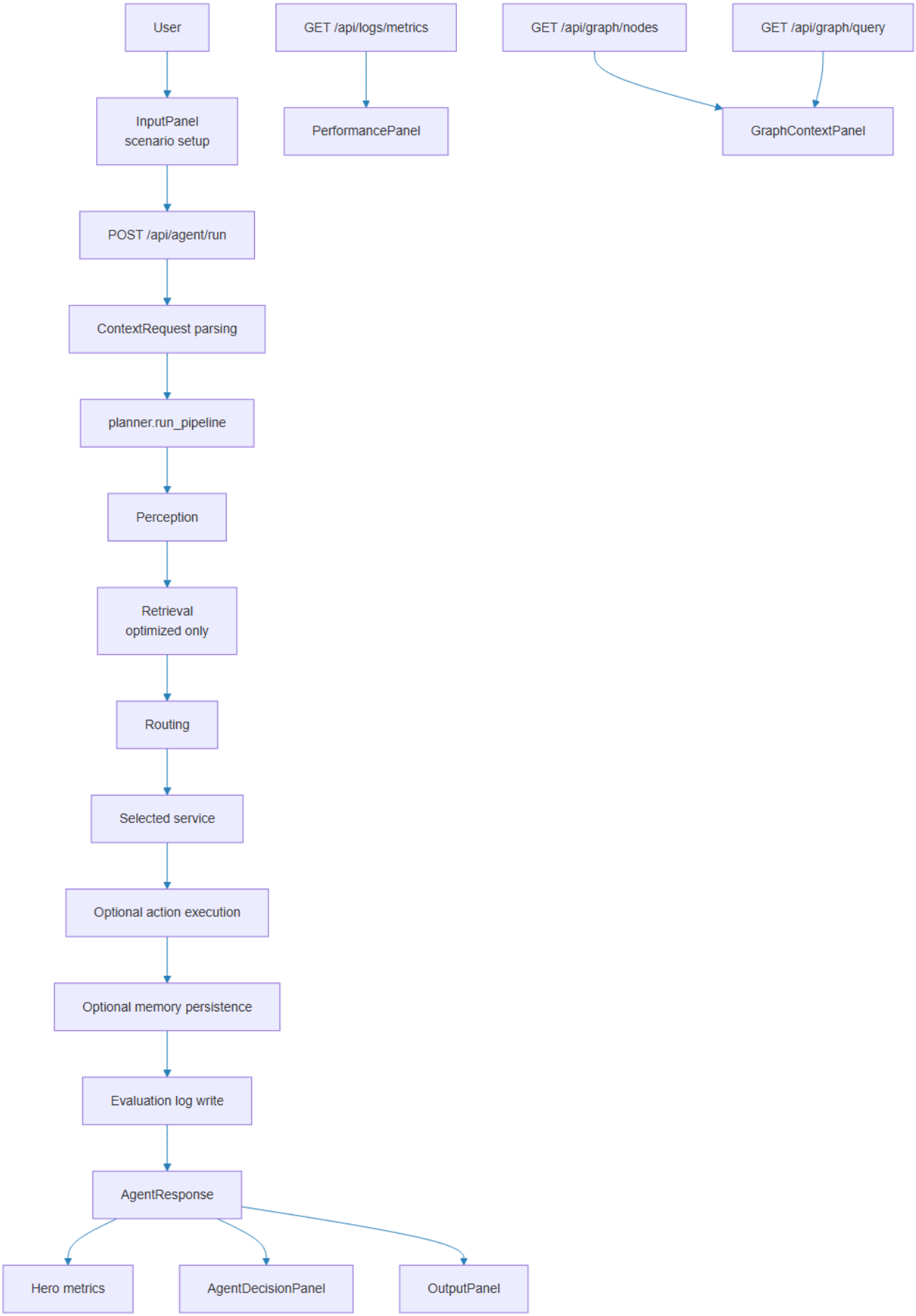
- smart_glasses_apps_api_walkthrough.md
- smart_glasses_user_request_data_flows.md
- smart_glasses_service_user_scenarios.md
- smart_glasses_user_request_data_flows_mermaid.md
- smart_glasses_user_request_sequence_diagrams.md
- smart_glasses_function_block_diagrams.md

The purpose of preserving those source files is traceability and reuse, while this document acts as the final integrated architecture reference.

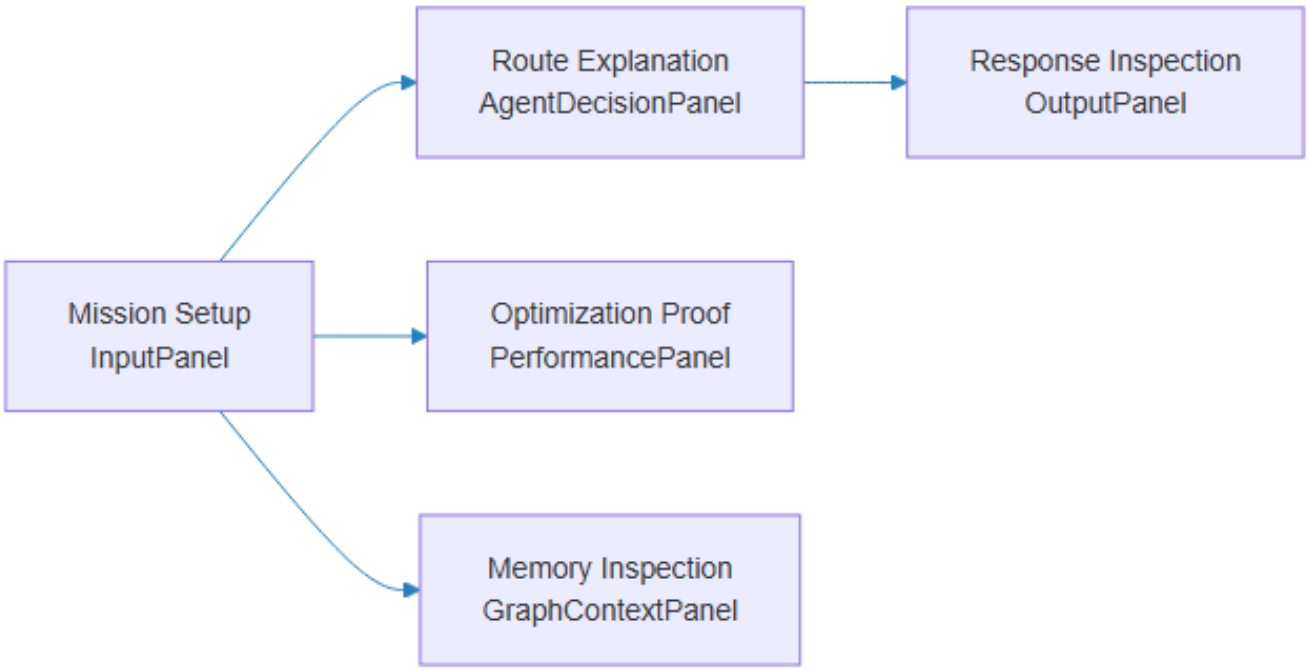
Appendix: Data Flow Diagrams

The following diagrams are rendered from the Mermaid sources in smart_glasses_user_request_data_flows_mermaid.md.

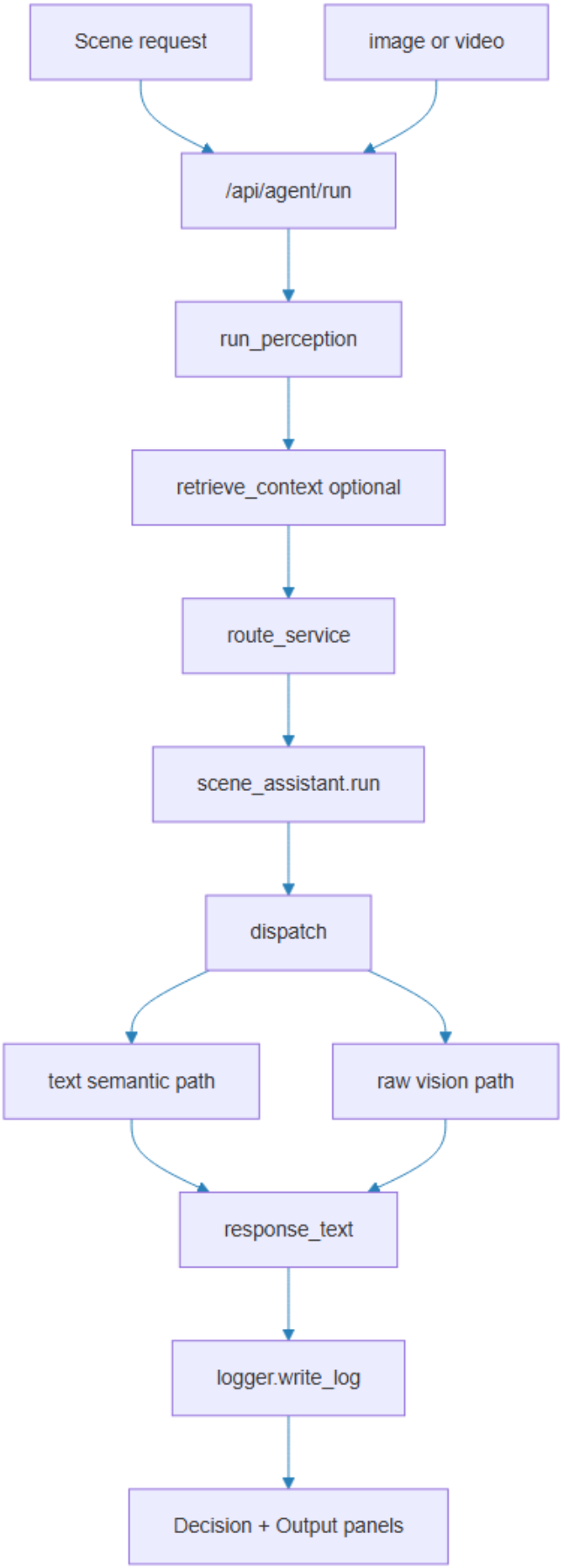
Common End To End Flow


User
InputPanel
scenario setup
POST /api/agent/run
ContextRequest parsing
planner.run_pipeline
Perception
Retrieval
optimized only
Routing
Selected service
Optional action execution
Optional memory persistence
Evaluation log write
AgentResponse
Hero metrics
AgentDecisionPanel
OutputPanel
GET /api/logs/metrics
PerformancePanel
GET /api/graph/nodes
GET /api/graph/query
GraphContextPanel

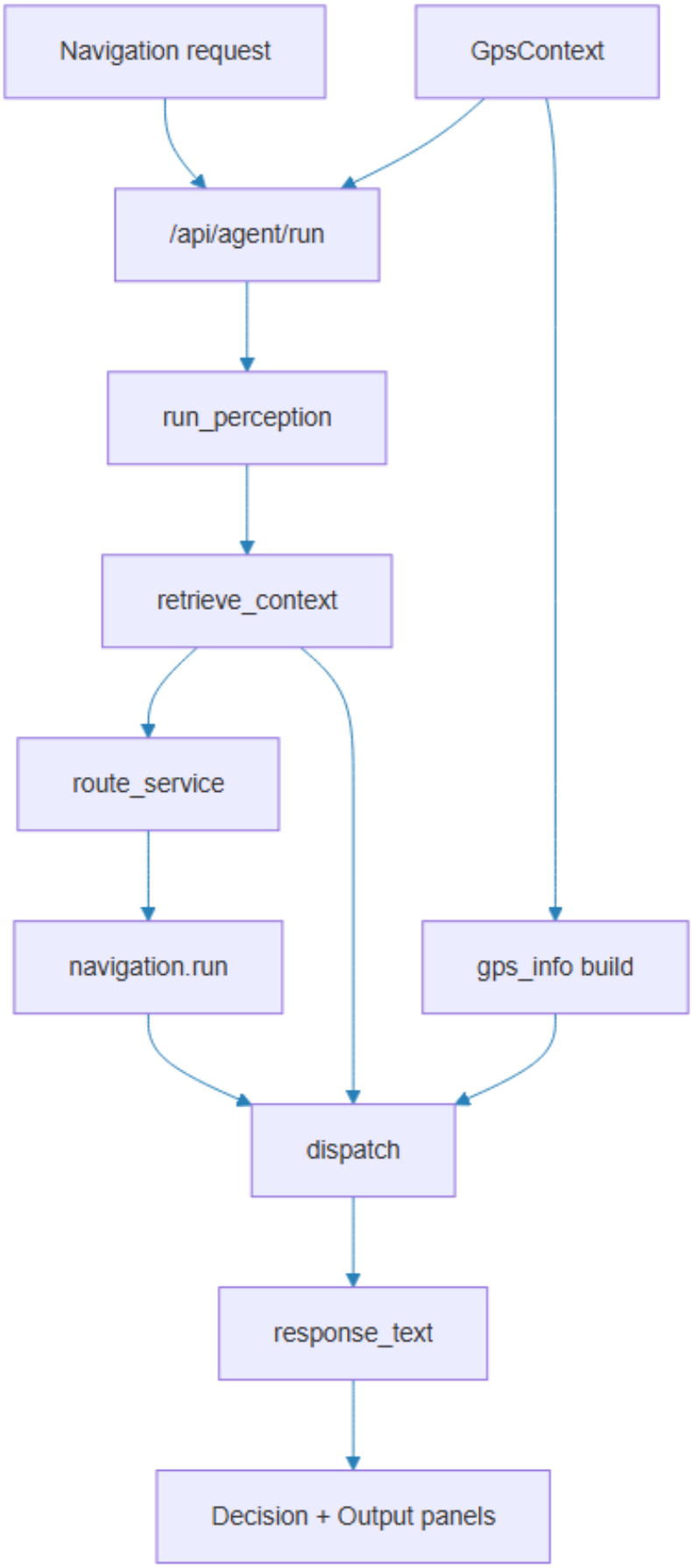
Operator Console Zones



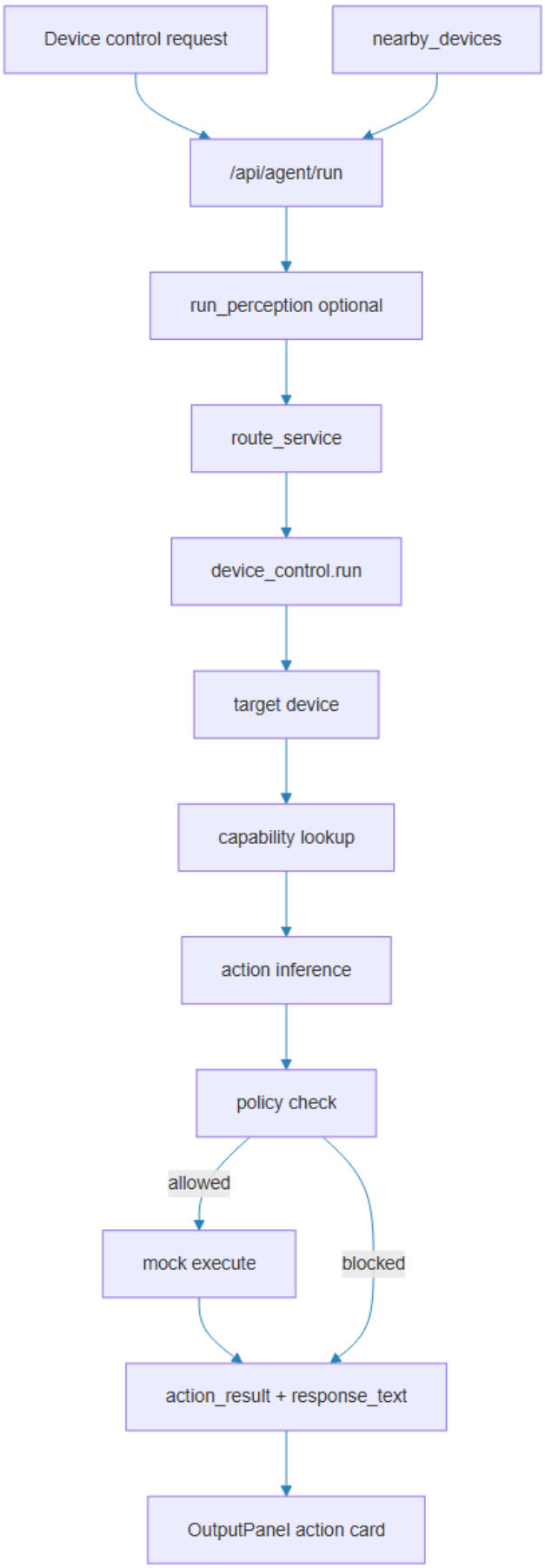
Scene Assistant Flow


Scene request
image or video
/api/agent/run
run_perception
retrieve_context optional
route_service
scene_assistant.run
dispatch
text semantic path
raw vision path
response_text
logger.write_log
Decision + Output panels

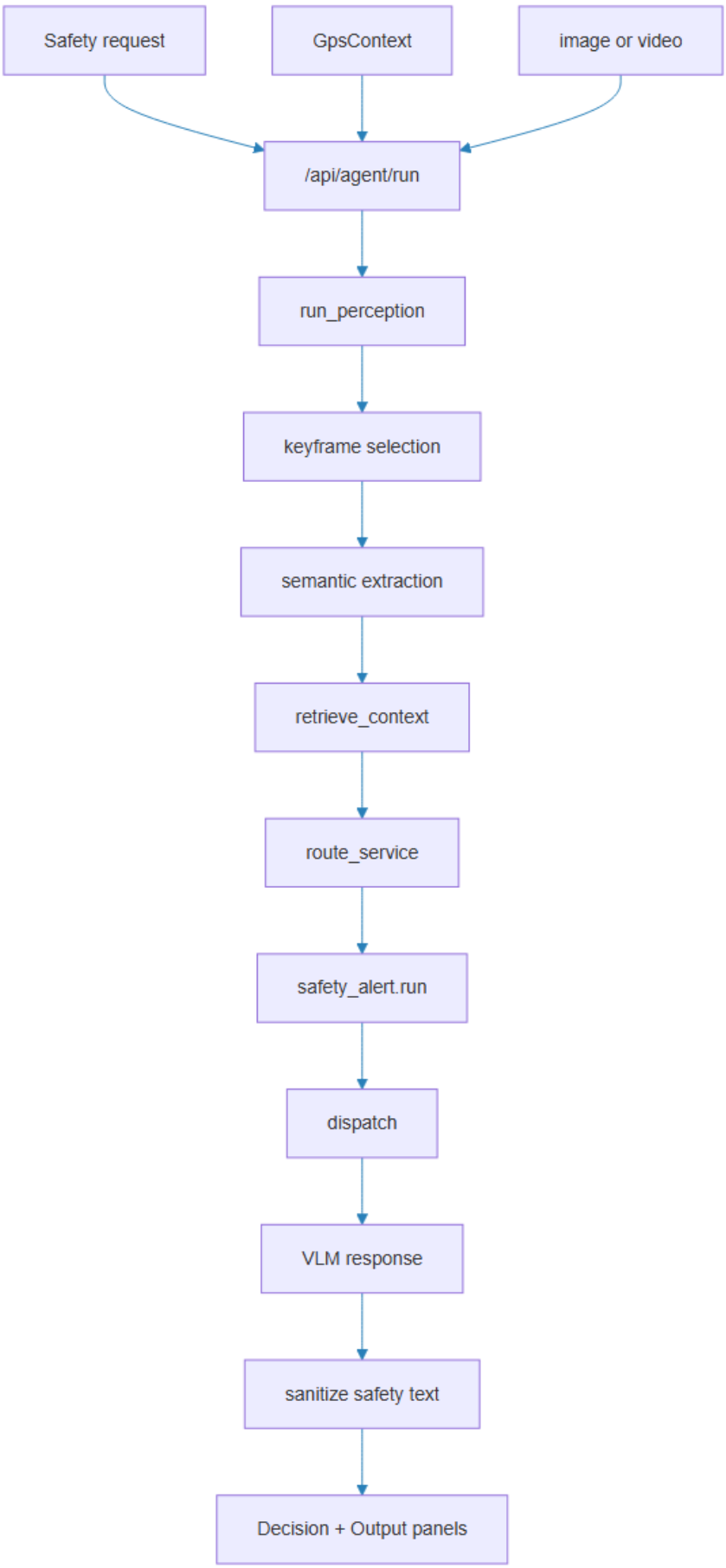
Navigation Flow


Navigation request
GpsContext
/api/agent/run
run_perception
retrieve_context
route_service
navigation.run
gps_info build
dispatch
response_text
Decision + Output panels

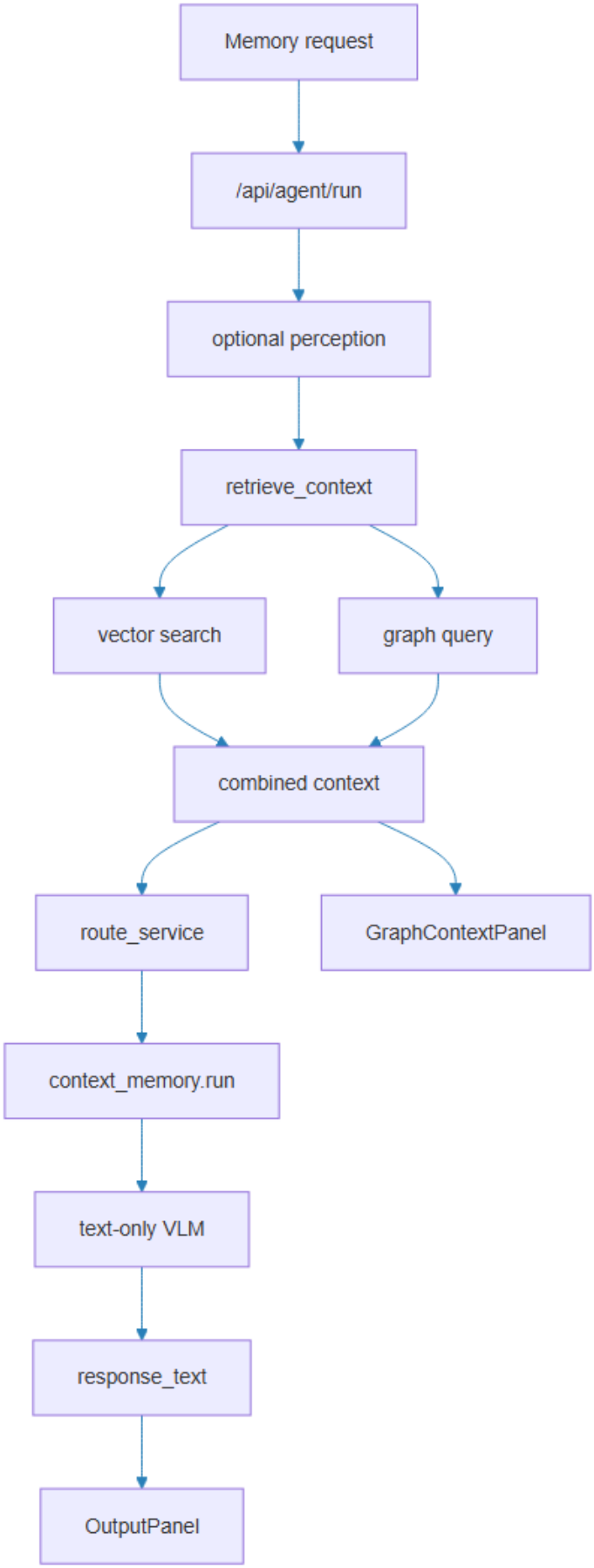
Device Control Flow


Device control request
nearby_devices
/api/agent/run
run_perception optional
route_service
device_control.run
target device
capability lookup
action inference
policy check
allowed
mock execute
blocked
action_result + response_text
OutputPanel action card

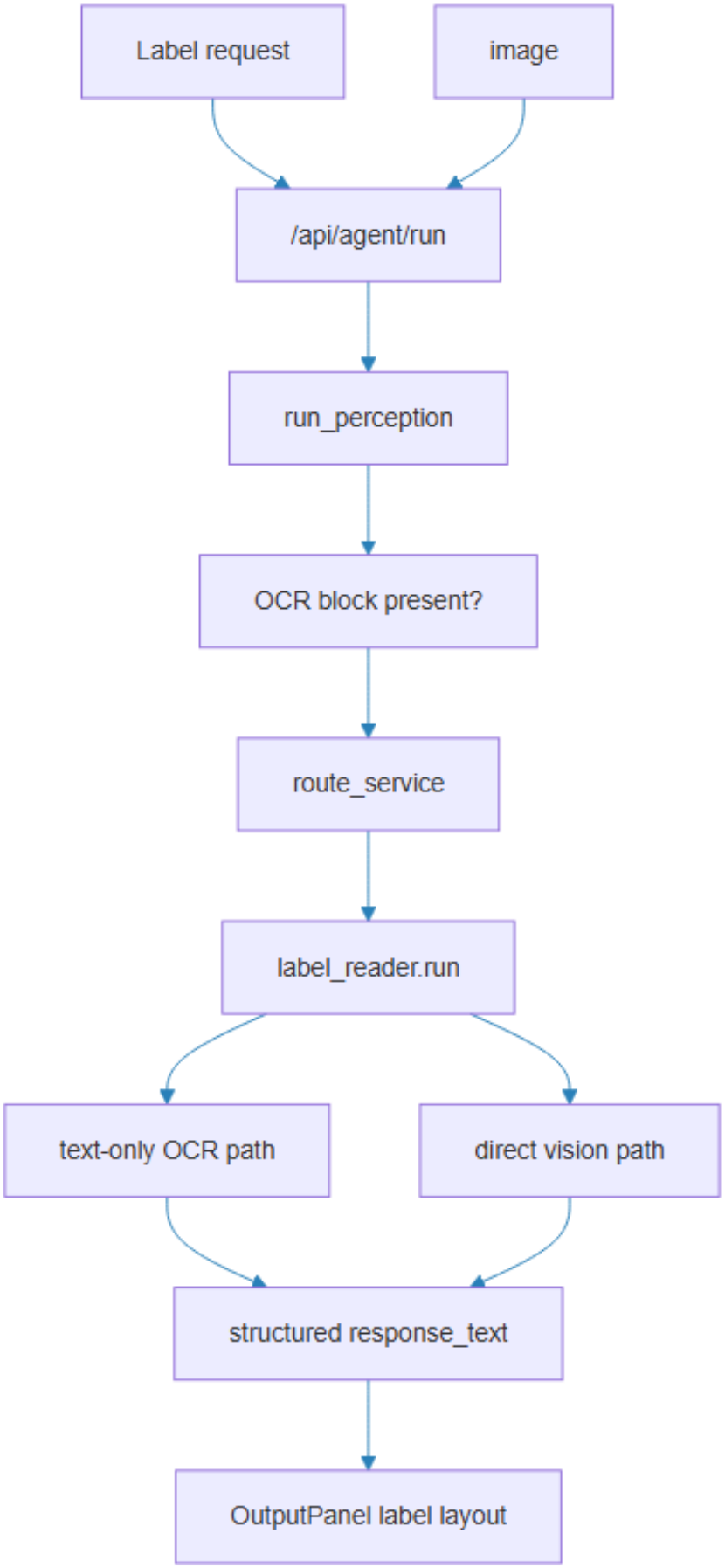
Safety Alert Flow


Safety request
GpsContext
image or video
/api/agent/run
run_perception
keyframe selection
semantic extraction
retrieve_context
route_service
safety_alert.run
dispatch
VLM response
sanitize safety text
Decision + Output panels

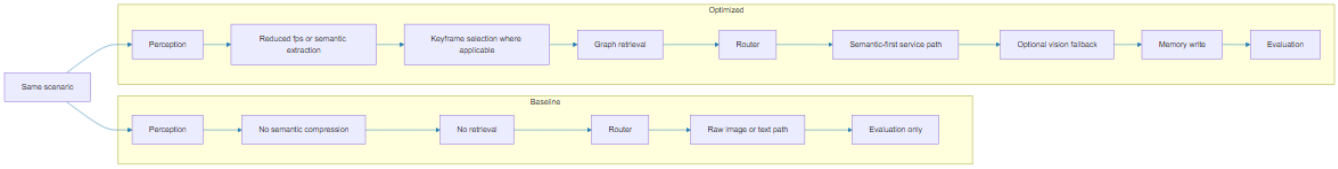
Context Memory Flow


Memory request
/api/agent/run
optional perception
retrieve_context
vector search
graph query
combined context
route_service
GraphContextPanel
context_memory.run
text-only VLM
response_text
OutputPanel

Label Reader Flow


Label request
image
/api/agent/run
run_perception
OCR block present?
route_service
label_reader.run
text-only OCR path
direct vision path
structured response_text
OutputPanel label layout

Baseline Vs Optimized Comparison



Evidence Fetch Flow